Avoiding Deadlock

Computer Systems

David Marchant

Based on slides by:

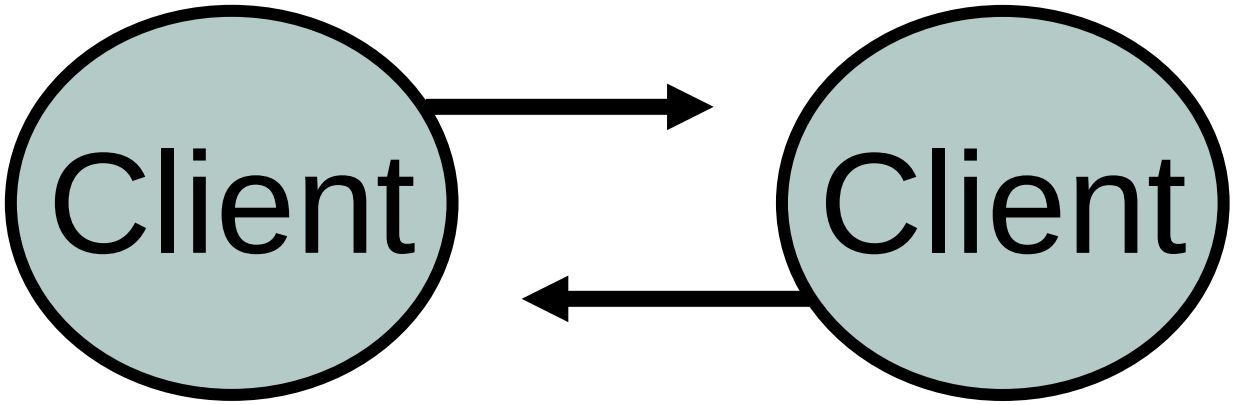
Randal E. Bryant and David R. O'Hallaron

Deadlock Avoidance

- **Deadlock can occur any time we have a blocking operation**
  - Network communications
  - Mutexes
- **Often time can be hidden in testing by buffers**
- **But if it CAN occur, we MUST assume it WILL**
- **Today we will look again at avoiding it locally, as well as across networks**

Deadlock isn't just local

- You cannot mutex over a network
- But you can have two hosts reading/writing which will act in the same way
- The client-server model is used entirely to escape this problem
- Communication Sequential Processes (CSP)



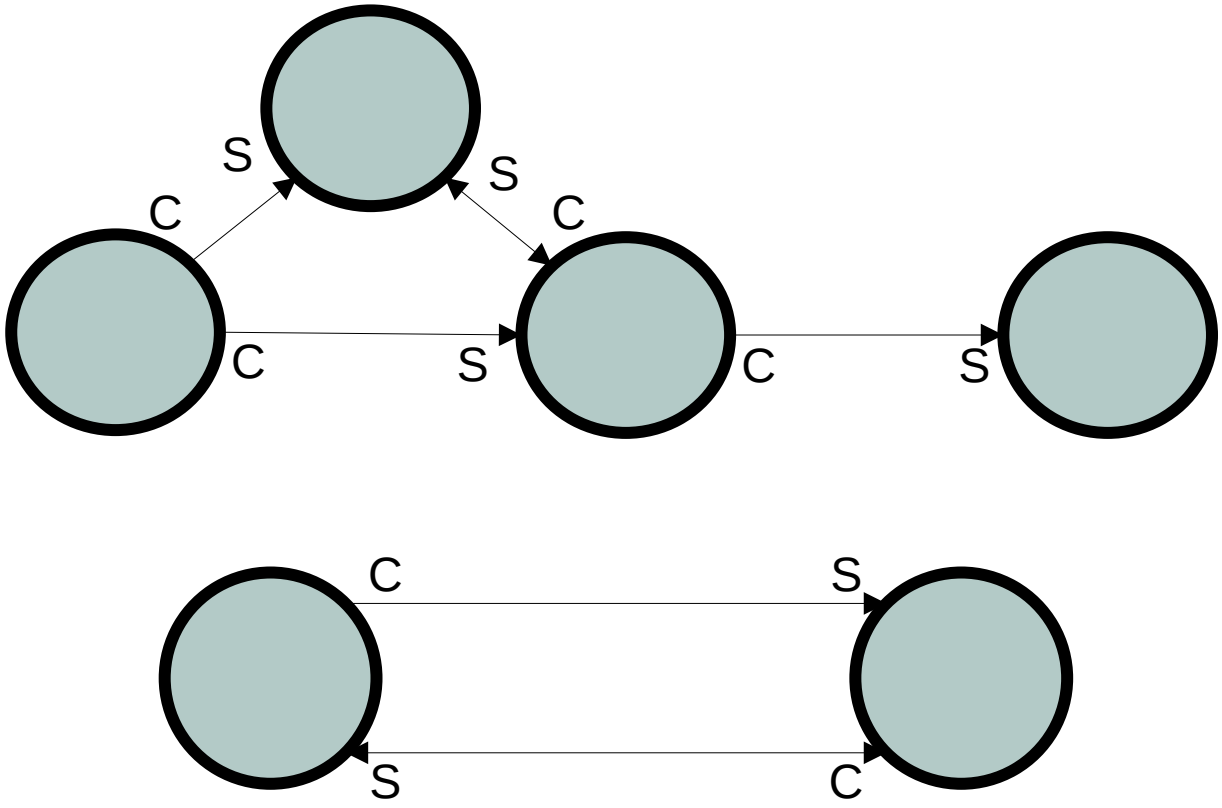
CSP

- **Communication Sequential Processes**
- **Proposed in 1978 by Tony Hoare**
- **Formal mathematical language for describing concurrent processes and their interactions**
  - Can *guarantee* no deadlock
  - Can *identify* livelock
- **Not a programming language, but principles are used in many contemporary languages such as Go**

Process Diagrams

- No formal definition for these diagrams or how they look
- Two components, processes and channels
- A process can represent an OS process, OS thread, network host, or any other sequential code
- A channel is a connection between processes, and may be mono- or bi-directional
- Can be helpful to label client and server ends of a channel

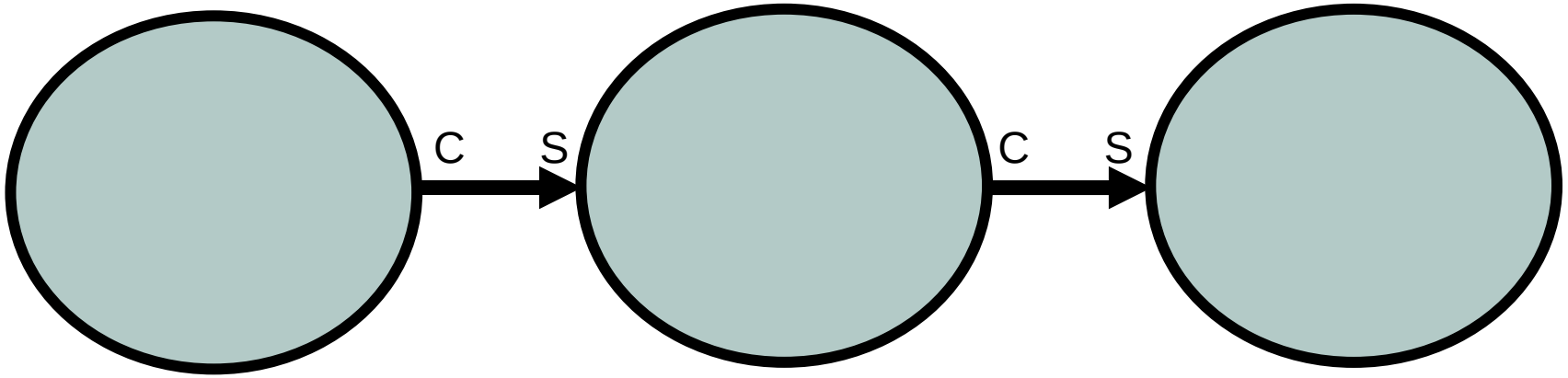
Process Diagrams



What does this get us

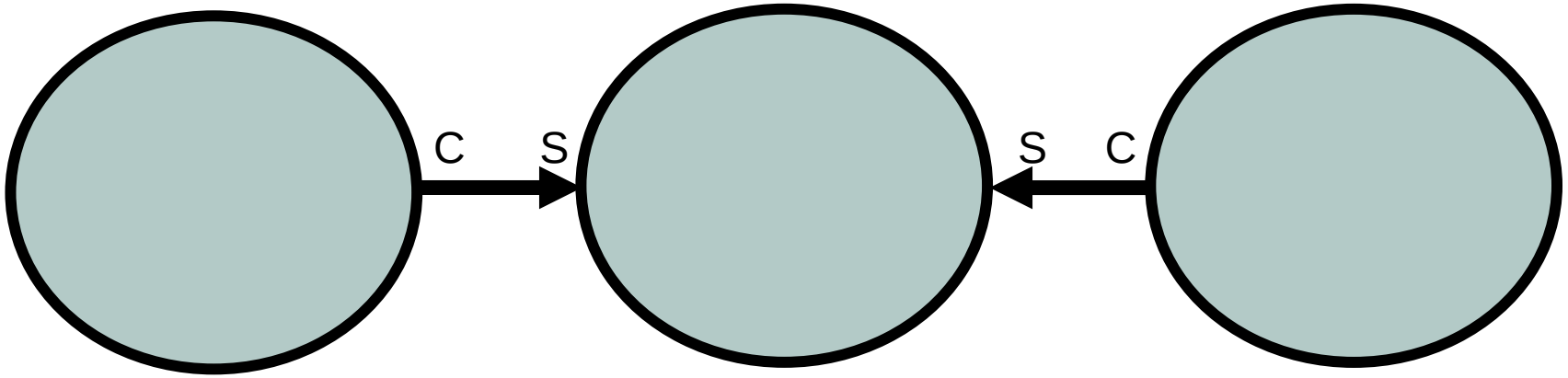
- **CSP is the source of the client-server model**
- **Semantically, in the client server model:**
  - Only clients initiate communications
  - If a client expects a response, a server will provide one in a finite amount of time
  - If a client expects a response, it will be immediately ready to receive it.
- **We have (hopefully) been keeping to this already**
- **If we can draw all channel interactions, we can understand all blocking points**
- **Any loop of client-server interactions has the potential to deadlock**

What about Deadlock between hosts?



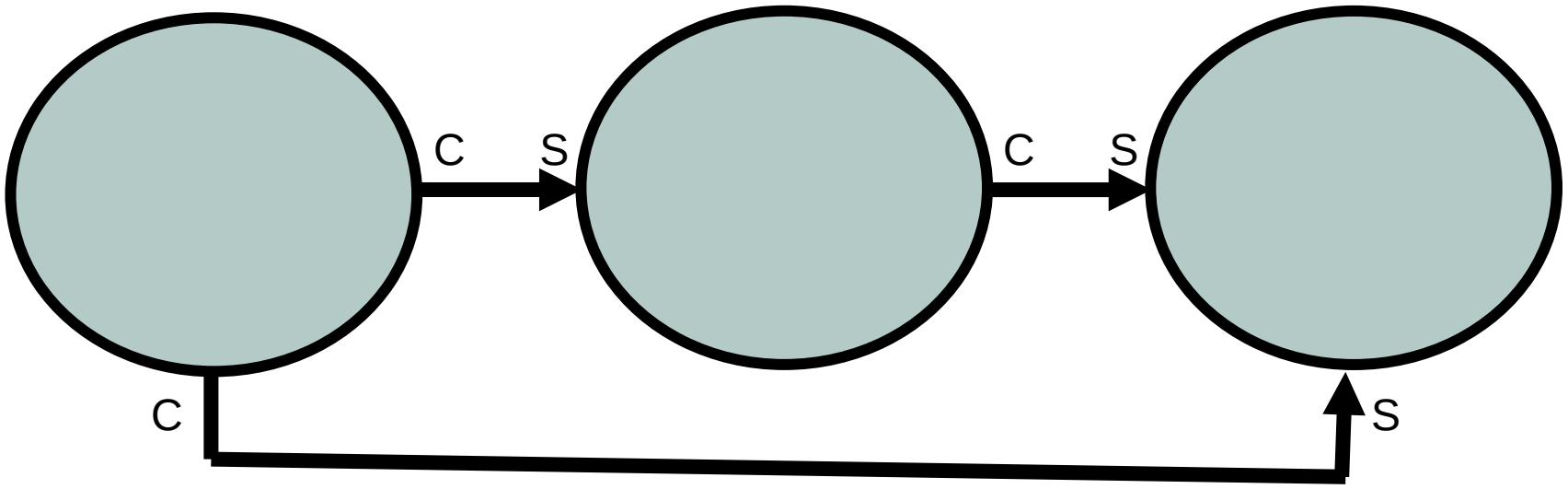
No deadlock

What about Deadlock between hosts?



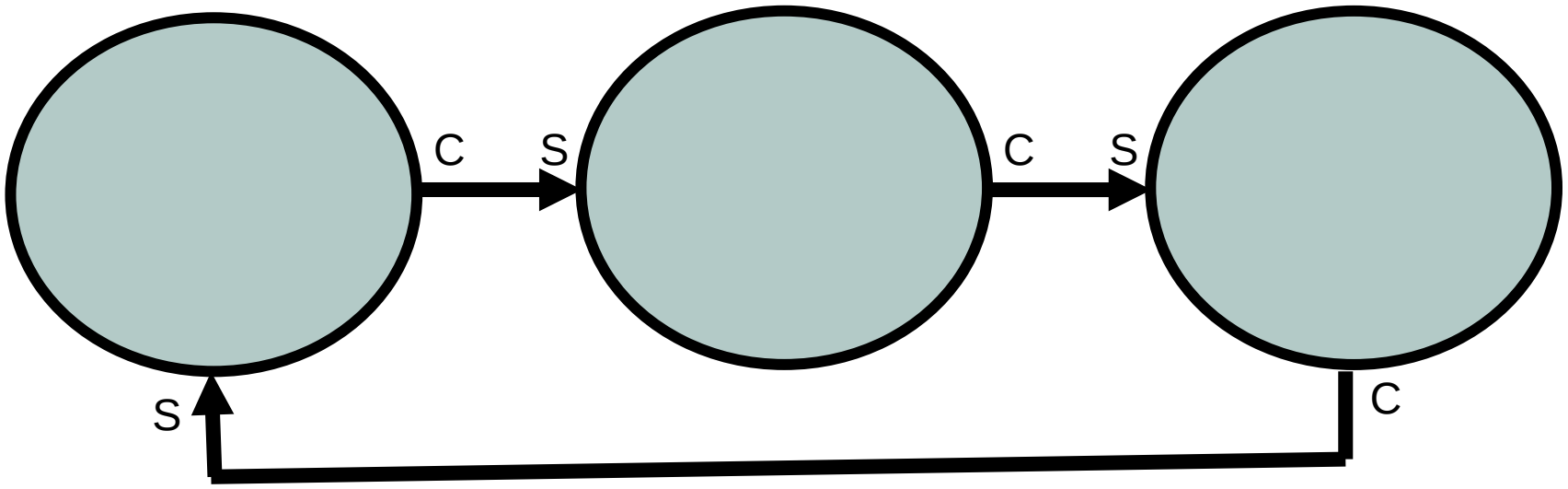
No deadlock

What about Deadlock between hosts?



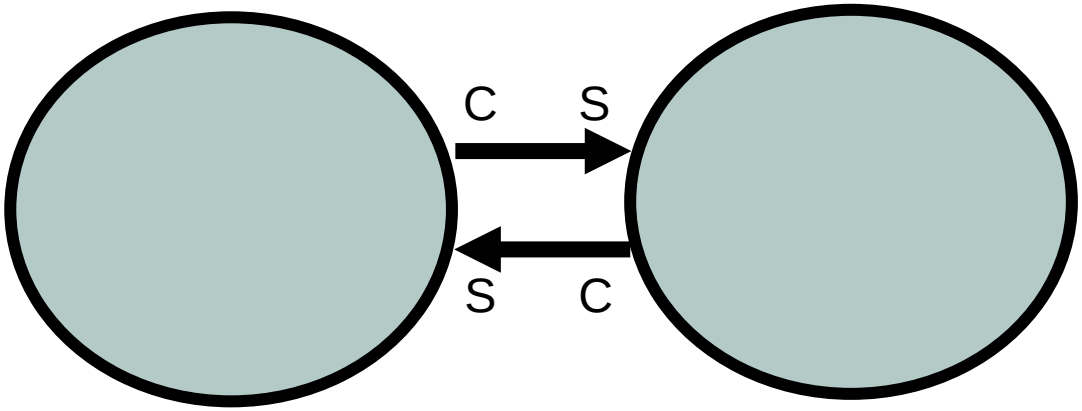
No deadlock

What about Deadlock between hosts?



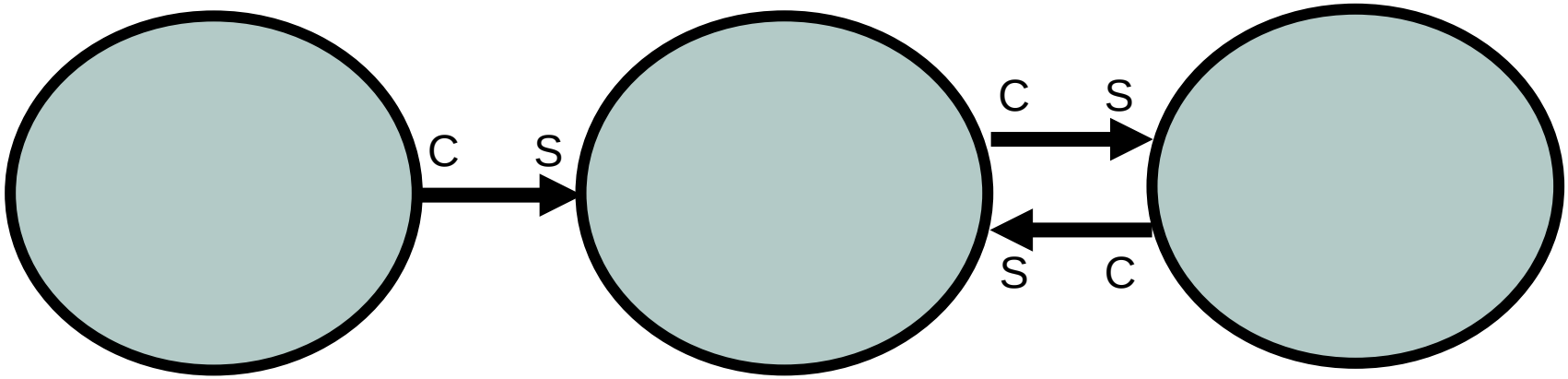
Deadlock

What about Deadlock between hosts?



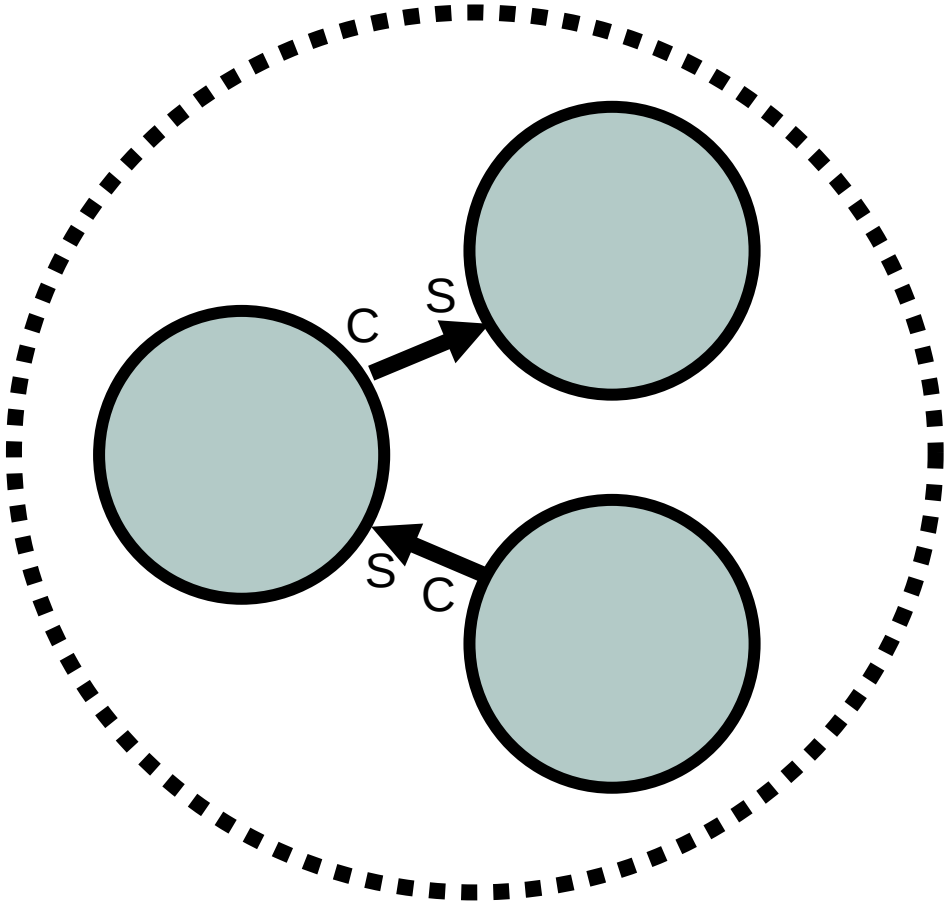
Deadlock

What about Deadlock between hosts?



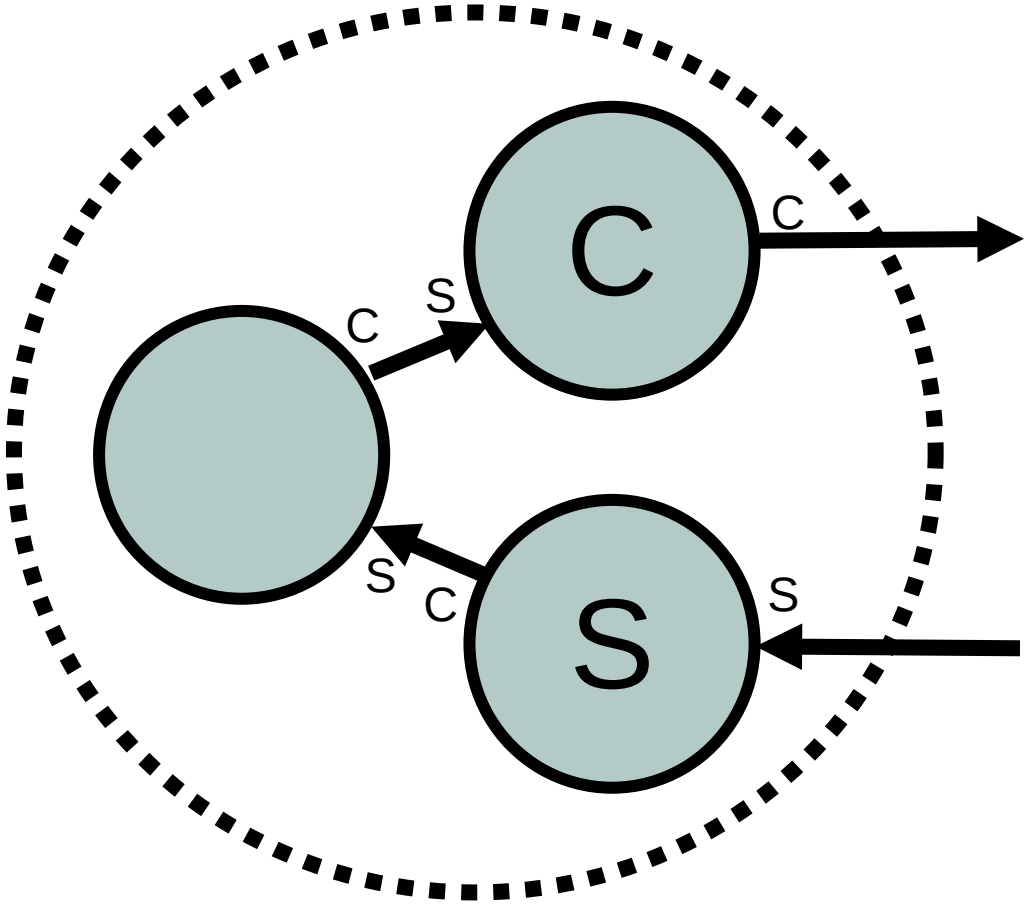
Deadlock

Didn't we solve this already?



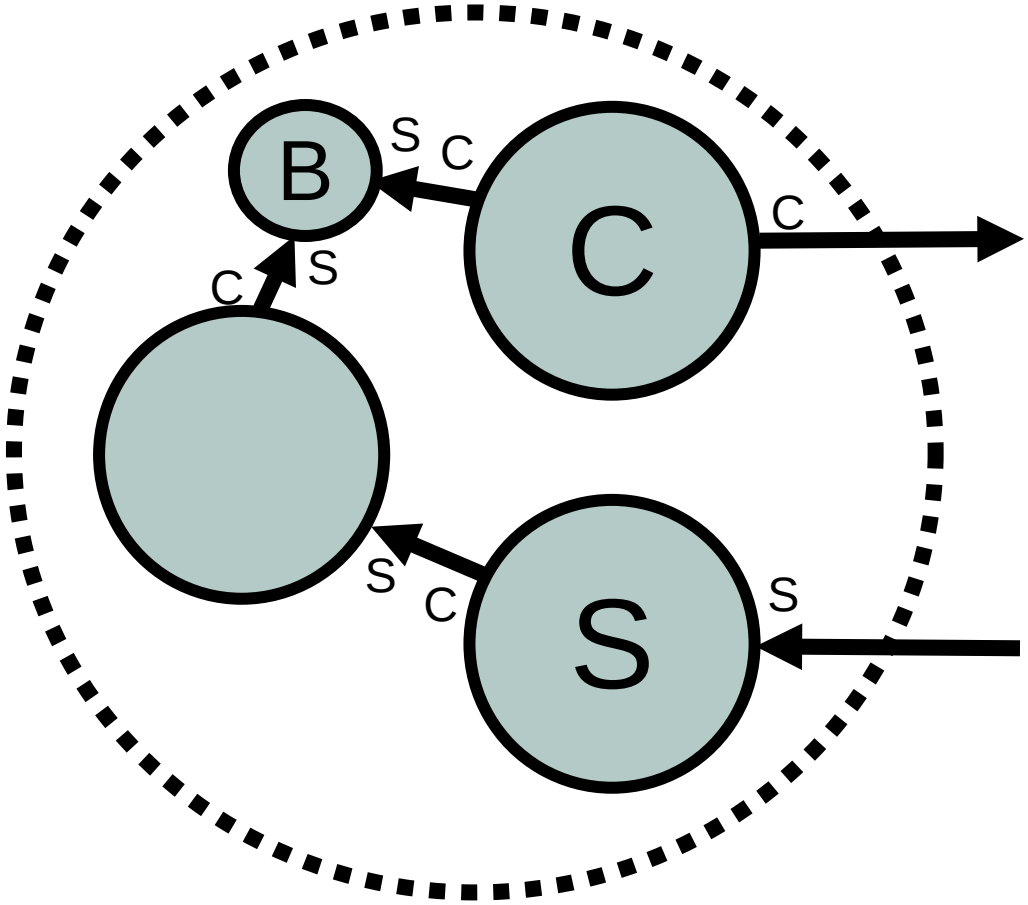
No deadlock

Didn't we solve this already?



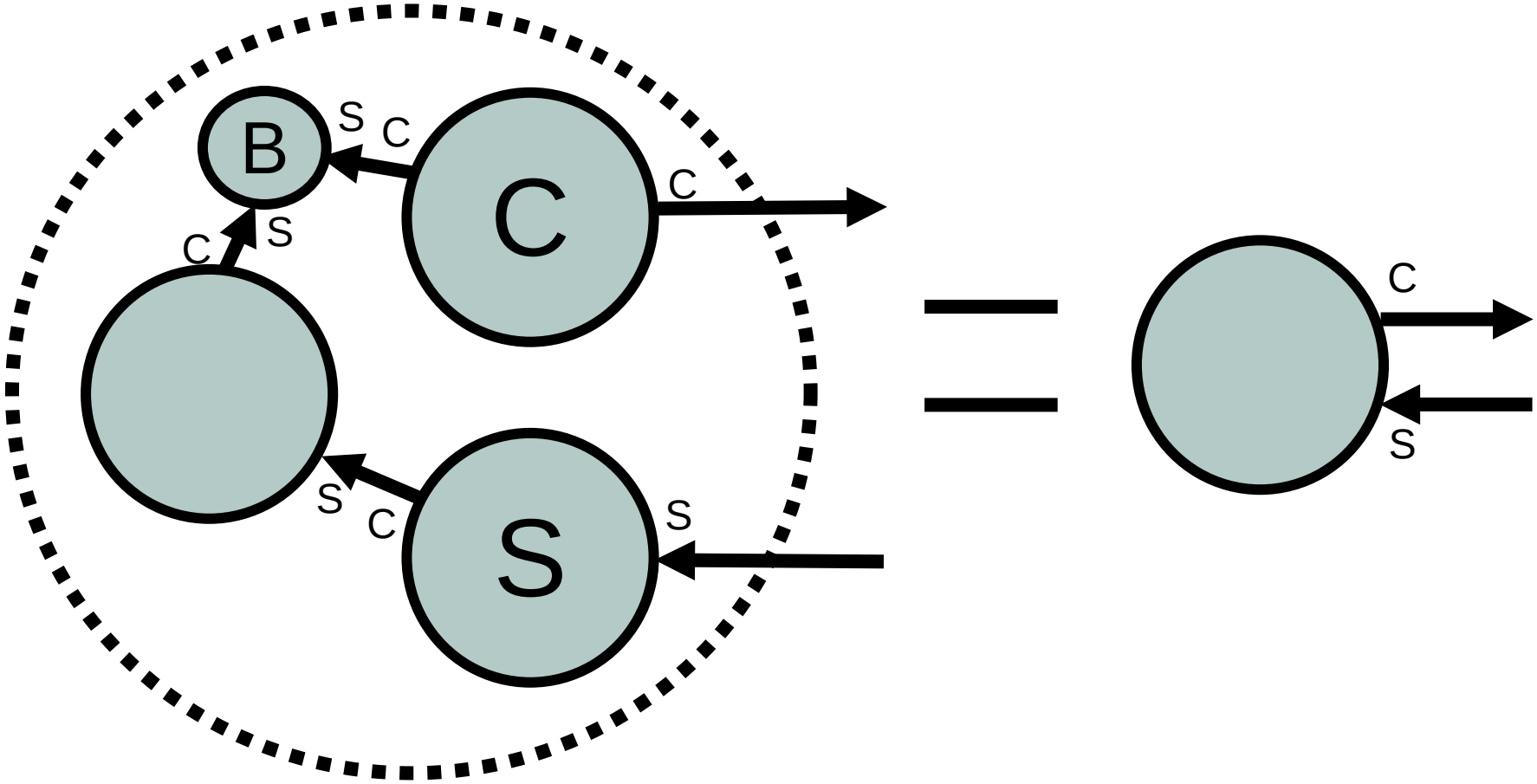
Maybe
Deadlock
deadlock?

Didn't we solve this already?



No deadlock

Why the dotted circle?



Some notes …

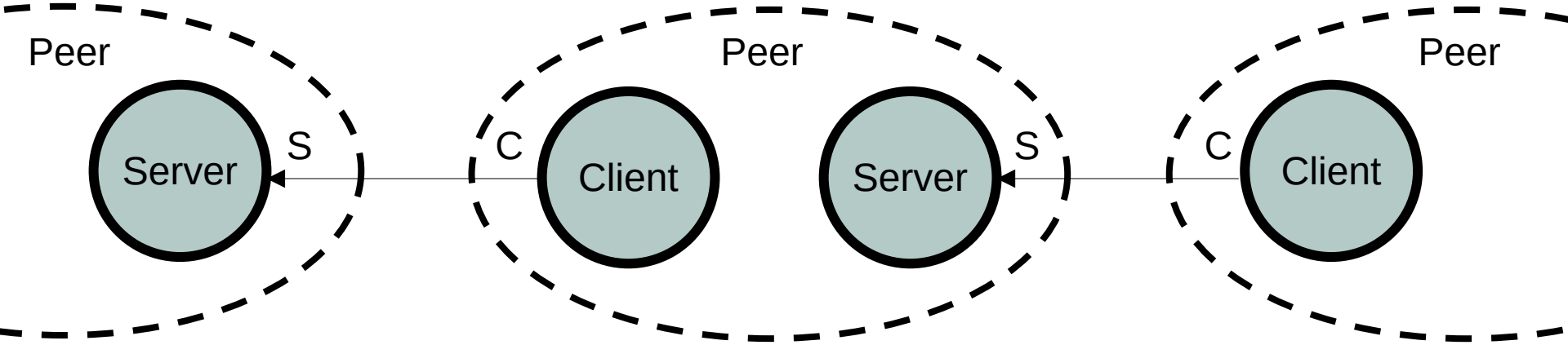
- A client/server loop doesn't actually mean deadlock
- But no client/server loop does guarantee no deadlock
- Depending on internal structure of a process, deadlock might not occur
- But the road to deadlock is paved by good intentions
- **Any client/server loop MUST be carefully examined and justified**

Translating code into diagrams

- These are more of a sketch, than a true reflection so some ambiguity is inevitable
- Only need to worry about blocking operations, external read and writes
- Concurrent connections should be shown as separate connections
- Sequential ones can be grouped
- Label channel ends (do as I say, not as I do)

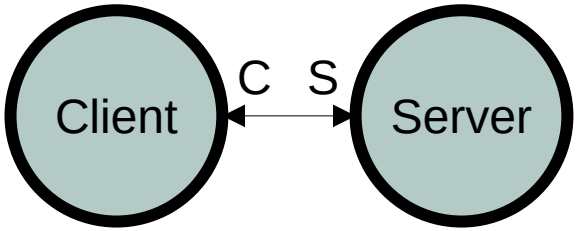
Translating code into diagrams

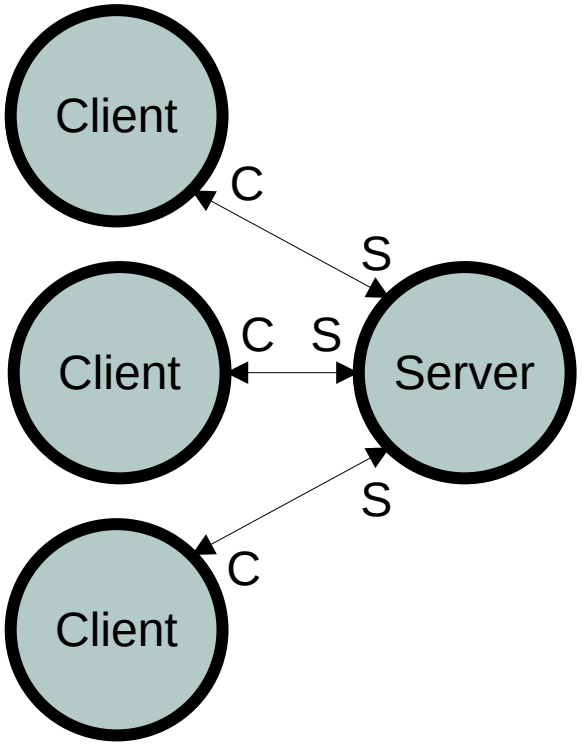
- **Lets draw A2**
- **A2 peers are all identical**
- **But have a server side and client**
- **A2 server interactions**
  - Listens for connections
  - Can handle parallel connections
  - Always responds
  - No additional comms from it
- **A2 Client interactions**
  - Connects to the others
  - Sends two message types (register, get, inform)
  - Each message is sequential



Translating code into diagrams

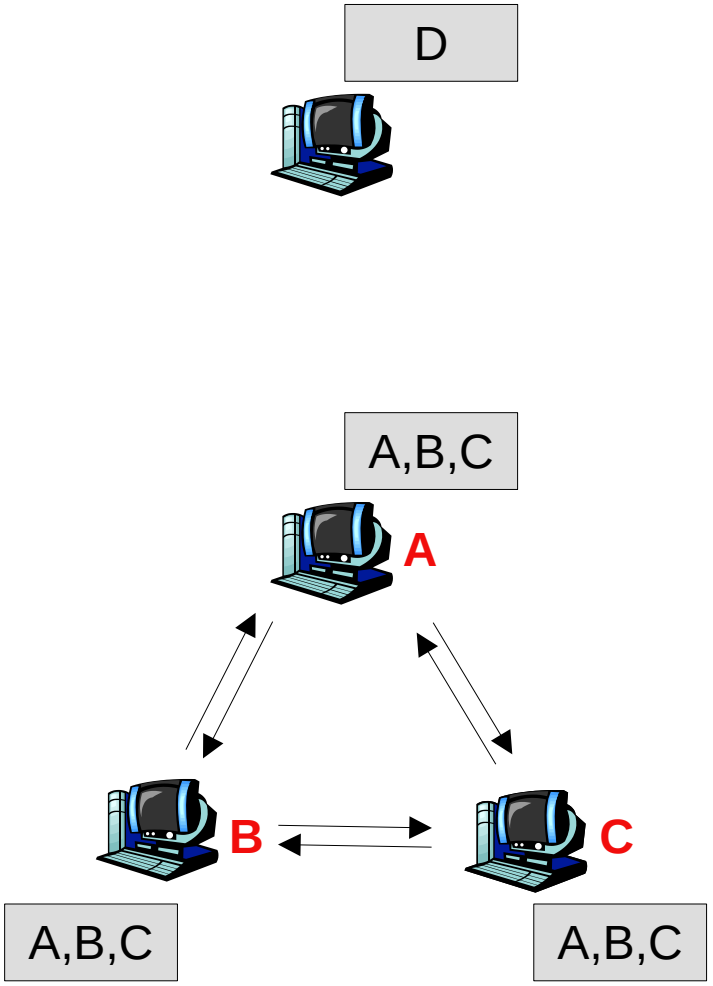
- Identical interactions can *often* be left out
- As each connection is served concurrently, we can effectively add infinite clients and nothing changes
- This might change if there are dependencies between connections





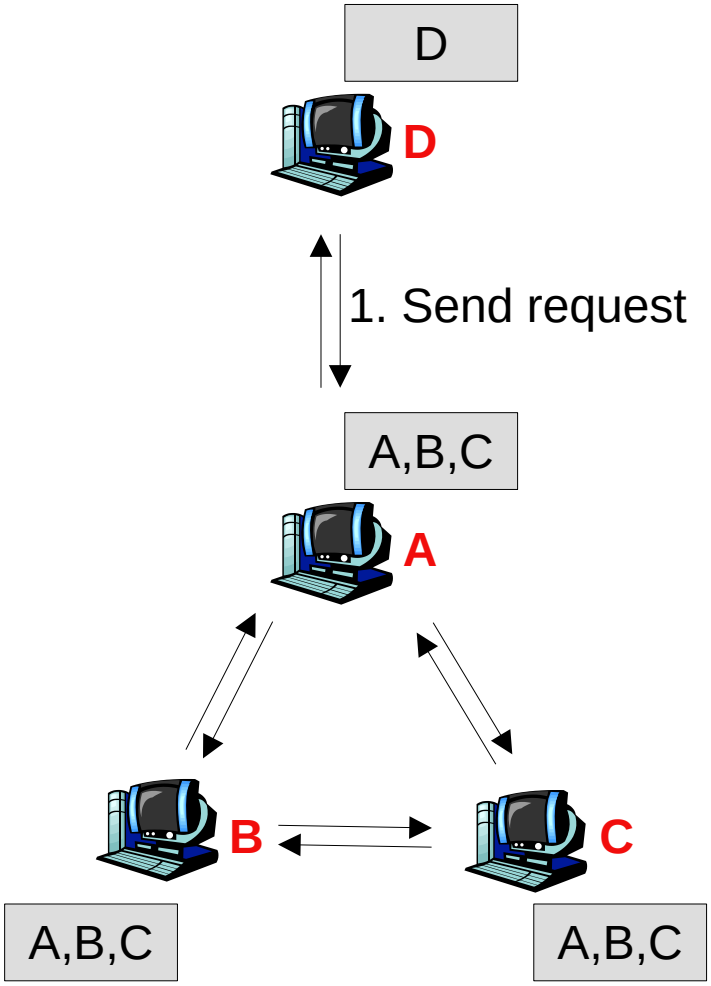
Peer to Peer Network in A2

- P2P is a way to share data files
- Peers connect to a network by registering with someone already on it
- Each Peer will attempt to maintain a list of everyone on the network
- If a peer gets a request to join, it will inform all the peers it knows about



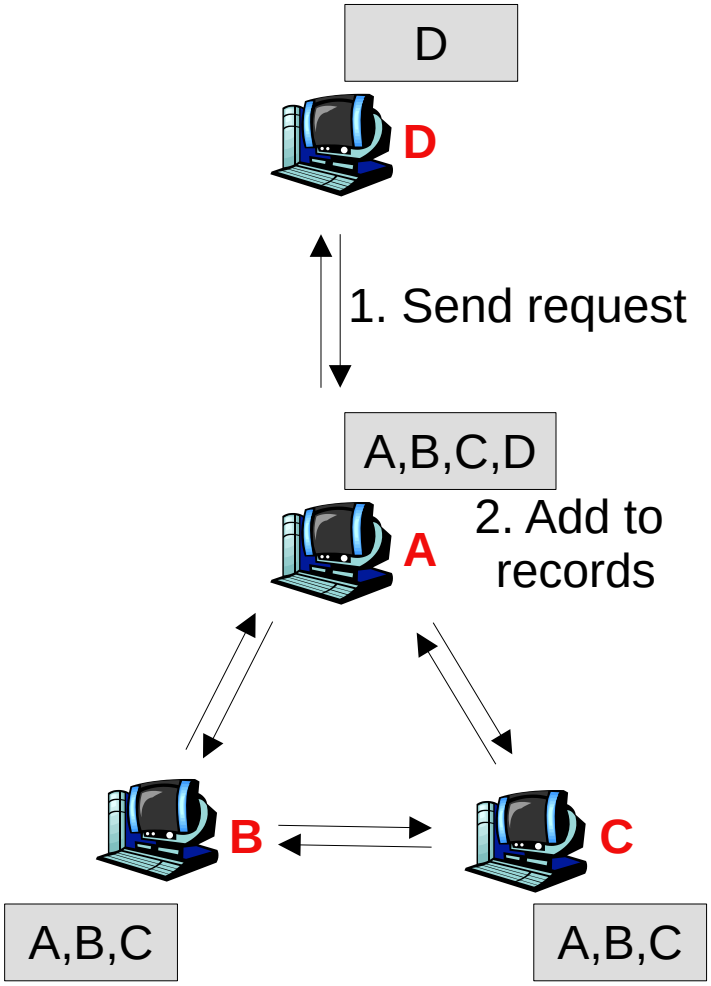
Peer to Peer Network in A2

- P2P is a way to share data files
- Peers connect to a network by registering with someone already on it
- Each Peer will attempt to maintain a list of everyone on the network
- If a peer gets a request to join, it will inform all the peers it knows about



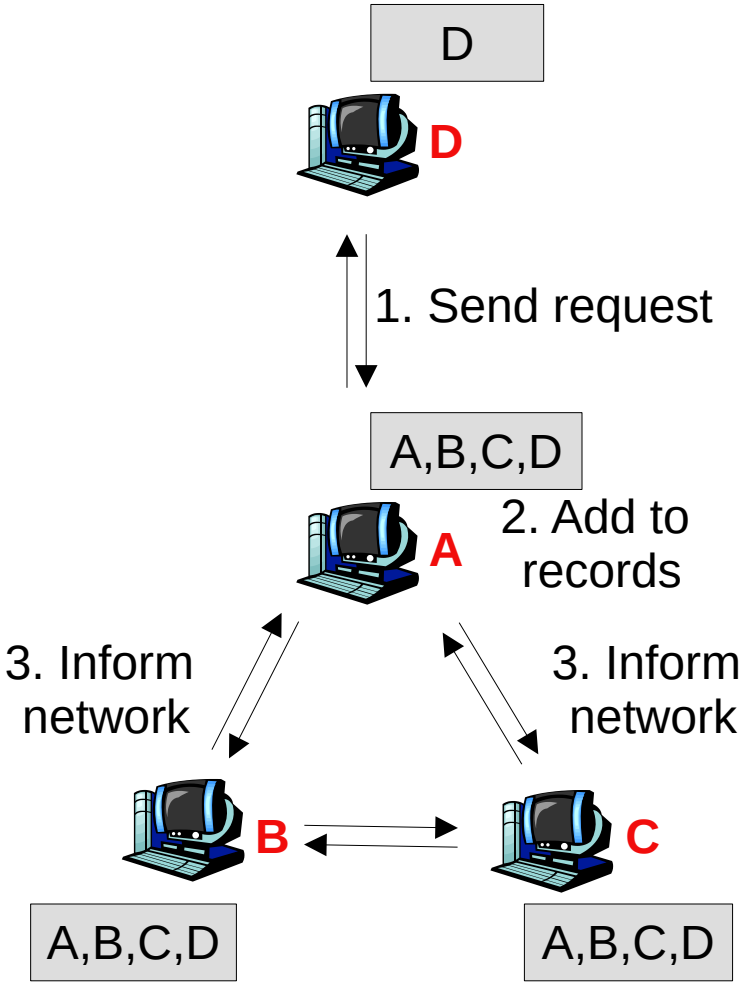
Peer to Peer Network in A2

- P2P is a way to share data files
- Peers connect to a network by registering with someone already on it
- Each Peer will attempt to maintain a list of everyone on the network
- If a peer gets a request to join, it will inform all the peers it knows about



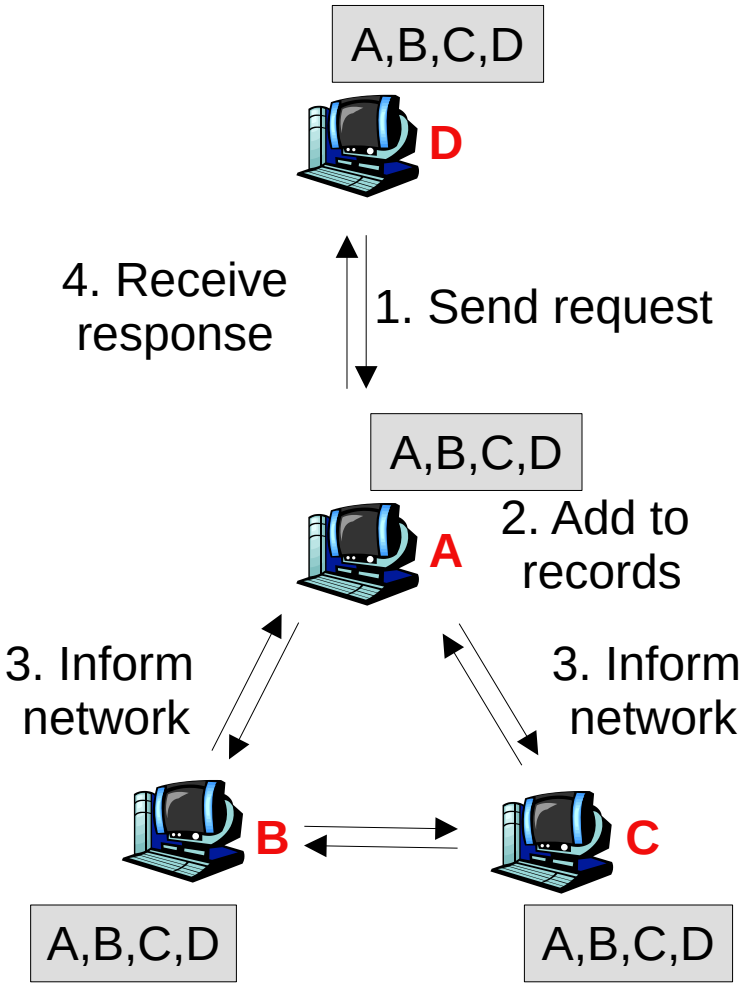
Peer to Peer Network in A2

- P2P is a way to share data files
- Peers connect to a network by registering with someone already on it
- Each Peer will attempt to maintain a list of everyone on the network
- If a peer gets a request to join, it will inform all the peers it knows about



Peer to Peer Network in A2

- P2P is a way to share data files
- Peers connect to a network by registering with someone already on it
- Each Peer will attempt to maintain a list of everyone on the network
- If a peer gets a request to join, it will inform all the peers it knows about



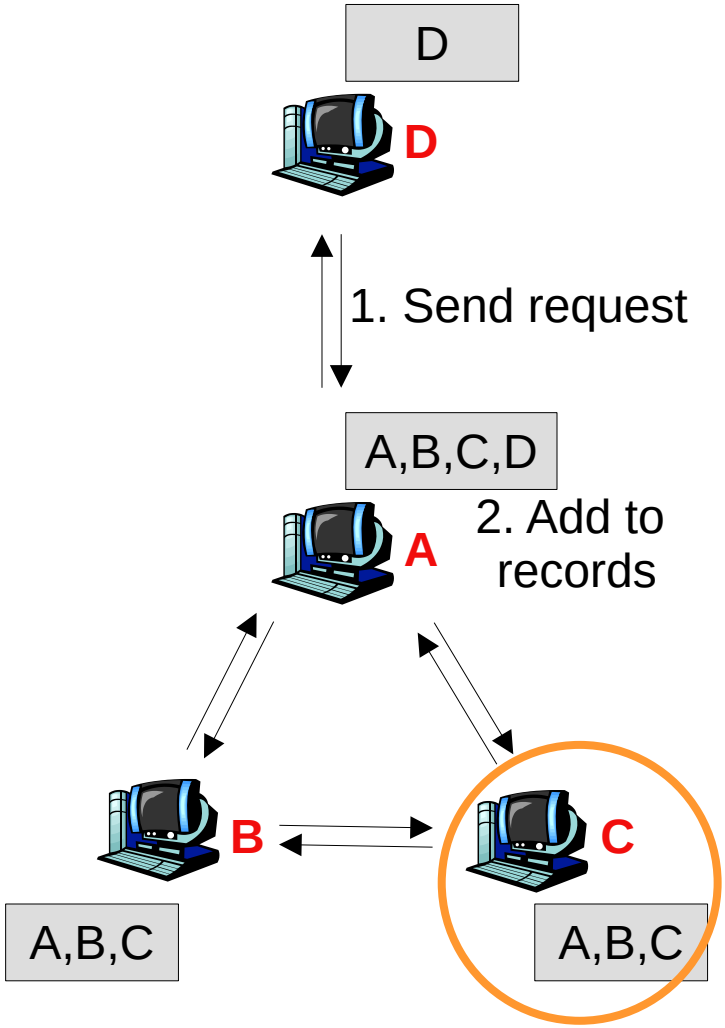
Races, across the network

- Recall that races occur when the outcome depends on the arbitrary ordering of interactions
- Fixed locally with locks, or by not sharing in the first place
- Global variable races don't occur as no global memory
- Locks do not exist at a network level (mostly)
  - Could centralise vital info, but this is sloooow
- Many (not all) races can be coped with
- Up to applications to avoid/cope with races as they occur

Races, across the network

- Consider if at this point, C wants to get a file, it only sees A and B as peers
- Race, as D *should* be included but is not yet
- Does this really matter though?
- For selecting a peer, maybe not
- If we needed a report of the complete network, maybe

- Solving this problem is out of scope, and can lead to lots of fun solutions (Santa problem)



Some conclusions

- Deadlock and races are as bad in networking as they are in multiprocessing/threading
- Races tend not to occur as no global memory
- Deadlock can very much occur both locally and remotely
- Use diagrams to debug the structure of your code
- A diagram is only useful if it reflects your implementation